

Golang Developer Assessment

Toy Blockchain and Ledger Simulator

A small system built from scratch in pure Go, with a short research report.

Assessment at a glance	
Document	Developer Requirements Specification
Role and level	Software Engineering Intern (Backend, Go)
Difficulty	Medium, with a genuine research component
Primary language	Go 1.22 or newer
Suggested timebox	5 working days, worked solo
Delivery format	Command-line application plus a written report
Version	2.0
Date	July 2026

1 Overview and Purpose

This assessment asks you to build a minimal blockchain, a small “toy” ledger, entirely in Go, and to accompany it with a short research report. There is no external blockchain, no network of peers, and no third-party chain library involved. Everything runs locally as a single command-line program.

The goal is not to test blockchain expertise. It is to see how you write Go, how you structure a small system from first principles, and above all, **how you reason about a problem and communicate your findings**. The blockchain is simply a compact, self-contained domain that exercises hashing, data structures, validation logic, and a clear success or failure condition. The research report is where we learn how you think.

HOW TO APPROACH THIS

Treat this as a scoped-down version of real work. Read the requirements, make sensible engineering decisions, document your reasoning, and ship something that runs. A smaller solution done well beats an ambitious one left broken.

2 What We Are Evaluating

In rough order of importance, this exercise is designed to reveal:

1. **Go fundamentals:** idiomatic structs, methods, error handling, slices and maps, packages, and clean use of the standard library.
2. **Problem decomposition:** turning a specification into well-named types and functions with clear responsibilities.
3. **Correctness and rigour:** deterministic hashing, working validation, and tests that prove your claims.
4. **Research and communication:** running your own small experiments and explaining what you found and why it matters.
5. **Engineering judgement:** knowing what to build, what to skip, and being honest about the limits of your solution.

3 Project Brief

Build a command-line program that maintains an append-only chain of blocks. Each block holds a small batch of transactions, is cryptographically linked to the block before it, and must satisfy a simple proof-of-work condition before it can be added. The program must be able to add transactions, mine new blocks, print the chain, and verify that the chain has not been tampered with.

Alongside the code, you will write a short report (see [Section 7](#)) in which you run a few experiments against your own implementation and discuss the results.

4 Scope

4.1 In scope

- A single-process Go application with a command-line interface.
- Block and chain data structures, deterministic SHA-256 hashing, and a genesis block.

- A basic transaction and ledger model with account balances.
- Proof-of-work mining with a configurable difficulty.
- Full-chain validation and tamper detection.
- Unit tests and a written research report.

4.2 Out of scope

- Networking, peer-to-peer gossip, or distributed consensus between nodes.
- Real cryptographic wallets, digital signatures, or key management (offered later as a stretch goal, not required).
- Smart contracts, a virtual machine, or any EVM or Solidity work.
- A web frontend or graphical interface. The command line is the interface.
- Any external blockchain, RPC node, or third-party chain SDK.

5 Functional Requirements

Priority key: **MUST** required **SHOULD** strongly expected **COULD** nice to have.

FR-1 Block structure **MUST**

Define a block type containing at minimum: a height or index, a Unix timestamp, the list of transactions it carries, the hash of the previous block, a nonce, and the block's own hash.

FR-2 Genesis block **MUST**

The chain must initialise with a single, deterministic genesis block whose previous-hash field is a fixed, well-known value such as all zeroes. All other blocks descend from it.

FR-3 Deterministic hashing **MUST**

A block's hash must be computed with SHA-256 over a stable serialisation of its fields, excluding the hash field itself. Hashing the same block twice must always yield the same result. Document exactly which fields feed the hash and in what order.

FR-4 Transactions and ledger **MUST**

Support a simple transaction model with a sender, a recipient, and an amount. Maintain account balances derived from the chain. Reject transactions that are malformed (a non-positive amount) or that spend more than the sender's balance. A special coinbase or faucet mechanism to introduce initial funds is acceptable and encouraged.

FR-5 Proof-of-work mining **MUST**

Adding a block requires finding a nonce such that the block's hash meets a difficulty target, for example a required number of leading zero hex digits. Difficulty must be configurable through a flag, constant, or config file. Mining should report the nonce found and, ideally, how long it took.

FR-6 Chain validation and tamper detection **MUST**

Provide a routine that validates the entire chain: every block's stored hash matches a recomputation, every previous-hash link is intact, every block satisfies the proof-of-work target, and heights and timestamps are consistent. Validation must return a clear pass or fail result and identify the first offending block on failure.

FR-7 Command-line interface MUST

Expose the core operations from the command line. At minimum: add a transaction to the pending pool, mine a block from the pending pool, print the chain in a readable form, validate the chain, and show account balances.

FR-8 Persistence SHOULD

Save the chain to disk, for example as JSON, and reload it on startup so state survives between runs. If you judge this out of reach in the timebox, say so in your report and keep the chain in memory.

FR-9 Configurable parameters COULD

Allow difficulty, block size (maximum transactions per block), and the data file path to be set through flags or a small config, with sensible defaults.

6 Non-Functional Requirements

- **Idiomatic Go.** Code must be gofmt-clean and pass go vet. Favour the standard library, and justify any third-party dependency.
- **Tested.** Include unit tests for hashing determinism, mining meeting the target, honest-chain validation, tamper detection, and transaction rejection.
- **Reproducible.** A fresh clone must build and run with documented commands. A Makefile or a few README lines is fine.
- **Tunable performance.** Difficulty must be low enough that mining a block finishes in seconds on a laptop. The reviewer should not wait minutes.
- **Readable.** Clear names, small functions, and comments where intent is not obvious, plus a concise README explaining your design choices.

7 Research Component

This section carries real weight. Write a short report of roughly **2 to 4 pages** (Markdown or PDF), driven by experiments you run against your own implementation. We care about clear reasoning and honest observations far more than length or polish.

7.1 Required investigations

6. **Tamper-evidence.** Deliberately alter a transaction in an early block, then run validation. Explain precisely why the chain becomes invalid and which check catches it. Include the before and after output.
7. **Difficulty versus effort.** Mine blocks at several difficulty levels and record the time taken and the number of hashes tried. Present the numbers in a small table and describe the trend. Is it linear, or does it grow faster? Why?
8. **Design write-up.** Explain your hashing scheme (which fields, in what order, and why) and how your validation guarantees integrity across the whole chain.

7.2 Discussion questions

- How does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in your local toy?

- Proof-of-work is one way to decide who may add the next block. Name at least one alternative, such as proof-of-stake or proof-of-authority, and give one advantage and one drawback versus proof-of-work in your own words.
- List three concrete ways your toy differs from a production blockchain (think about consensus among peers, transaction signatures, Merkle trees, and finality). Pick one and sketch how you would add it.

Cite any sources you rely on. We are testing your ability to research and synthesise, not to memorise.

8 Technical Constraints and Guidelines

- Go 1.22 or newer. The standard library is strongly preferred; crypto/sha256, encoding/json, and time cover most needs.
- No external blockchain, RPC endpoint, or chain SDK. This is a from-scratch exercise.
- Structure the code into clear packages or files, for example block, chain, ledger, and cli, rather than one large main.go.
- Use of AI assistants is permitted, but you must understand and be able to explain every line you submit. Expect a short walkthrough conversation.
- Commit your work in small, meaningful steps. We like to see how the solution evolved.

9 Acceptance Criteria

The submission is accepted when the following scenarios hold. They are written in Gherkin style and map directly to the functional requirements.

Scenario: Chain starts from a deterministic genesis block (FR-2)

Given a freshly initialised blockchain
When the chain is created
Then it contains exactly one block at height 0
And that block's previous-hash equals the fixed genesis value

Scenario: Block hashing is deterministic (FR-3)

Given a block with fixed fields and a fixed nonce
When its hash is computed twice
Then both hashes are identical

Scenario: A mined block satisfies the difficulty target (FR-5)

Given a difficulty of N leading zero digits
When a new block is mined
Then the block's hash begins with at least N zeros
And the found nonce reproduces that exact hash

Scenario: An honest chain validates successfully (FR-6)

Given a chain of several mined blocks

When the chain is validated
Then validation reports the chain as valid

Scenario: Tampering with a block is detected (FR-6)

Given a valid chain of several blocks
When a transaction inside an earlier block is modified
And the chain is validated
Then validation fails
And it identifies the first block whose hash no longer matches

Scenario: An overspending transaction is rejected (FR-4)

Given an account whose balance is 100
When a transaction attempts to send 150 from that account
Then the transaction is rejected
And the account balance is unchanged

10 Deliverables

9. **Source code** in a Git repository or a zipped archive that builds and runs.
10. **README** with build and run instructions, a list of command-line commands, and a short note on your design decisions and any known limitations.
11. **Tests** runnable via `go test ./...`
12. **Research report** in Markdown or PDF covering Section 7.

11 Evaluation Rubric

Your submission is scored across the following areas.

Area	Weight	What good looks like
Core correctness: hashing, mining, validation, ledger	30%	Mechanics work and validation truly catches tampering.
Go code quality and structure	20%	Idiomatic, well organised, gofmt and vet clean.
Research report	20%	Real experiments, clear reasoning, honest limits.
Tests	15%	Cover the claims that matter, and actually pass.
CLI and documentation	10%	Easy to run; README explains the design.
Initiative and stretch goals	5%	Thoughtful extras that add real value.

12 Suggested Milestones

A guide, not a mandate. Organise the five days however suits you.

Day	Focus
Day 1	Design the types; implement the block, genesis, and deterministic hashing.
Day 2	Build the chain, add proof-of-work mining, and tune difficulty.
Day 3	Add the transaction and ledger model, plus full-chain validation.
Day 4	Write tests, build the CLI, add persistence, and run the experiments.
Day 5	Write the research report, polish the README, and submit.

13 Stretch Goals (Optional)

Attempt these only after the core is solid. Any one of them is a strong signal, and none are required.

- **Digital signatures.** Sign transactions with a key pair and verify them during validation.
- **Merkle root.** Summarise a block's transactions with a Merkle root instead of hashing the raw list.
- **Concurrent mining.** Search the nonce space across goroutines and stop cleanly when one succeeds, a natural fit for Go.
- **Difficulty retargeting.** Adjust difficulty automatically to keep block times roughly constant.
- **Fork resolution.** Accept competing chains and implement a longest-valid-chain rule.

14 Submission

Share a link to a private repository, or send a zipped archive, containing all deliverables. Make sure a clean checkout builds and runs using only the documented commands. Be ready for a short call where we ask you to walk through your design, run a scenario or two, and talk about what you would do next with more time.